

Internet Programming — Laboratory 5

Juliusz Chroboczek

28 October 2025

Exercise 1.

1. Write a Go program that downloads a file from the URL given as a command-line parameter and saves it to disk. Use the `flags` package for parsing the command-line.
2. Modify your program so that it downloads the file by making sequential range requests for pieces of the size given by the `-c` command-line option (default 16 kB). The syntax of range requests is defined in RFC 7233 Section 3.1, and you will find examples on MDN¹. Make sure that your program behaves correctly if the server doesn't obey range requests (replies with a 200 result instead of 206). Test your program both against a server that obeys range requests and one that doesn't.
3. How much slower is piece-wise downloading? Why does it depend on the location of the server?
4. Modify your program so that it uses a `If-Range` header to avoid inconsistencies:
 - if the server provides a (strong) e-tag on the first reply, then use the e-tag in the `If-Range` header;
 - if the server provides an `If-Modified-Since` header on the first reply, then use the value of the header in the `If-Range` header;
 - otherwise, fall back to a full download.

You will find the syntax of the `If-Range` header in RFC 7233 Section 3.2, and examples on MDN.

Exercise 2. Modify your program so that it downloads multiple pieces in parallel. Your program will first download the first piece. If the server provides neither a (strong) e-tag nor a last-modified time, perform a full download. Otherwise, download the file piecewise using n simultaneous threads (*goroutines*), where n is the value of the command-line parameter `-n` (default 4).

You may structure your program as follows:

- a main program, that fetches the first piece then produces a stream of piece descriptions to download which it sends over a channel;
- n threads, that compete to read a piece description from the channel.

1. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Range>

The main program may signal that there are no pieces left to download by closing the channel. You may use the `(*File).WriteAt` method to avoid concurrency issues when writing data to disk. You may use a `sync.WaitGroup` in order to determine when the worker threads have terminated.

Verify that your program does use the expected number of connections, using either `tcpdump`, *Wireshark*, or simply `netstat`.

Exercise 3. Modify your program so that if the server replies with a redirect to the first request, it performs the subsequent requests on the target of the redirect.